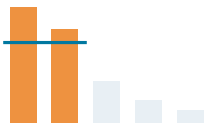
Fast Projection onto the Top-k-Sum Constraint

Find two cutoff values instead of sorting millions of numbers

Main idea introduction



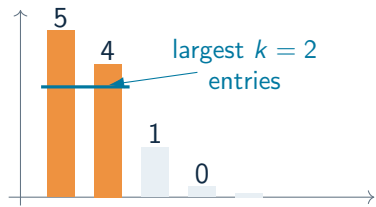
cap only the largest k entries

Projection means the smallest possible repair

The task

Given a vector $\boldsymbol{a}$, find the nearest vector $\boldsymbol{x}$ such that

$T_k(\boldsymbol{x}) :=$ sum of the largest k entries of $\boldsymbol{x} \leq r$.



Objective: make the smallest repair that keeps the biggest k values under a budget.

Many models care about the biggest few values

Risk control

Limit the largest losses rather than every small fluctuation.

Robust modeling

Penalize the worst few components without crushing the rest.

Matrix optimization

The same geometry appears in Ky Fan norm constraints on singular values.

Why speed matters

Projection is often an inner step in a larger optimization loop. A slow projection can become the cost that everyone pays repeatedly.

A tiny example: move only what causes the violation

$\boldsymbol{a} = (5, 4, 1, 0), \qquad k = 2, \qquad r = 6.$

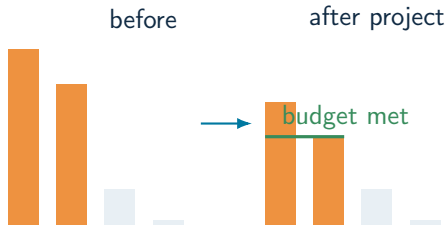
The largest two entries sum to

$$5 + 4 = 9 > 6.$$

A nearest feasible point is

$$\boldsymbol{x}^\star = (3.5, 2.5, 1, 0).$$

$$3.5 + 2.5 = 6.$$



The hidden cost is ranking the data

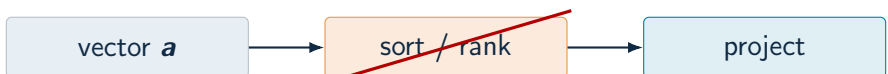
Existing exact methods often need the order of the entries to identify which values determine the top-k budget.

That usually means:

- partially sorting to check whether the constraint is already satisfied;
- sorting to compute the projection in the active case;
- moving large amounts of data before the real correction begins.

The paper's question

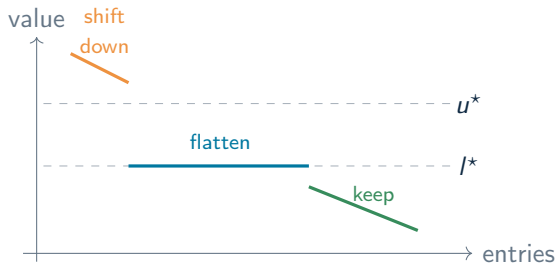
Can we recover the same exact projection without first sorting the whole vector?



The geometry: two thresholds are enough

Once two numbers $u^\star$ and $l^\star$ are known, the projection has a simple shape:

$$x_i^\star = \begin{cases} a_i - (u^\star - l^\star), & a_i > u^\star, \\ l^\star, & l^\star \le a_i \le u^\star, \\ a_i, & a_i < l^\star. \end{cases}$$



So the real task is not finding every $x_i^\star$ one by one.

The real task is finding the two thresholds.

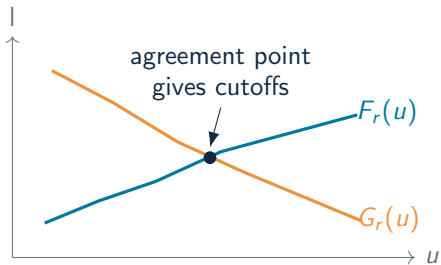
The trick is to make the cutoffs agree

The optimality equations know the answer, but the usual form asks for ranked entries.

We rewrite them as two threshold formulas that can be evaluated by unsorted scans.

$$F_r(u) = \frac{1}{k}\left(r - \sum_i \max(a_i - u, 0)\right),$$

The second formula, $G_r(u)$, gives the lower cutoff implied by the same trial upper cutoff u.



In the relevant case, the cutoffs are found by solving

$$F_r(u) = G_r(u).$$

EIPS searches, filters, and finishes exactly

1. Start smart

Pick an initial interval where the crossing can live.

2. Search and filter

Evaluate the formulas by scans, discard entries that cannot affect the cutoff, and reuse sums.

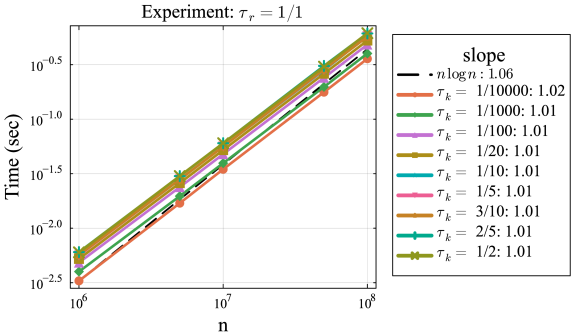
3. Refine exactly

Once the active piece is known, solve the remaining linear equation and recover the two cutoffs.

Main computational win

EIPS avoids explicitly sorting the full vector while still returning the exact Euclidean projection.

Experiments show faster runtimes when sorting is the bottleneck



What the experiments show

- Runtime grows roughly linearly in the tested regimes.
- The speed advantage grows when sorting dominates the baseline.
- One active-case setting at $n = 10^7$ reports about 17× faster than ESGS and about 4000× faster than Gurobi.

Careful claim

The paper proves finite termination. The near-$O(n)$ runtime is an empirical observation, not the worst-case guarantee.

What to remember

Geometry

The projection is governed by two thresholds: shift, flatten, keep.

Rewriting

Ranked quantities are replaced with formulas that can be evaluated by unsorted scans.

Algorithm

EIPS searches for a curve crossing instead of sorting the full vector.

Fast exact projection by finding cutoffs, not by sorting.